# PocketSmart AI: Your Smart Budget & Recommendation Assistant

## Project Description

Managing budgets across different life needs—like home decor, event planning, or jewelry shopping—can be overwhelming due to the wide variety of products, platforms, and price ranges. PocketSmart AI addresses this challenge through a GenAI-powered, cross-platform recommendation system that delivers personalized, budget-based suggestions for products and services.Using Gemini 1.5 Flash Pro, PocketSmart AI analyzes user preferences, budgets, and contextual needs to generate curated recommendations from popular platforms such as Amazon, Flipkart, IKEA, Swiggy, Zomato, OYO, and more. The platform helps users plan interior designs, parties, and even select matching jewelry for special occasions—all within their defined budgets.PocketSmart AI transforms budgeting into an intelligent, user-friendly experience through its smart planners, adaptive forms, and recommendation engine that bridges multiple categories and e-commerce ecosystems.

## Scenario 1: Home Interior Planning with Smart Budget Allocation

Users begin by selecting the Home Budget Planner. After entering their budget, they specify room types (e.g., Living Room, Kitchen, Bedroom) and quantities for lights, ceiling fans, dining tables, etc. Gemini 1.5 Flash Pro processes this data and recommends cost-effective options for each category across platforms like IKEA and Amazon.Recommendations are balanced across functionality, style, and price, ensuring users can decorate their spaces beautifully without overspending.

## Scenario 2: AI-Based Party Budget Planning

In the Party Planner, users input their total budget, guest count, event type, and venue details. Based on these inputs, the AI allocates the budget proportionally across catering, decoration, and entertainment, sourcing options from vendors listed on Swiggy, Zomato, and even accommodation services like OYO.The system tailors suggestions based on event type (e.g., birthday, corporate, wedding), helping users organize a successful event without financial guesswork.

## Scenario 3: Jewelry Recommendations for Occasions

The Jewelry Budget Planner allows users to enter a budget, select the occasion, and define style preferences. Users can optionally upload an outfit image for the AI to consider color coordination and aesthetics. Gemini 1.5 Flash Pro analyzes the data and offers jewelry options from platforms like Amazon and Flipkart, tailored to occasion and style.This ensures that users receive elegant, matching jewelry suggestions that fit both their personal style and financial plan.

## Architecture Overview

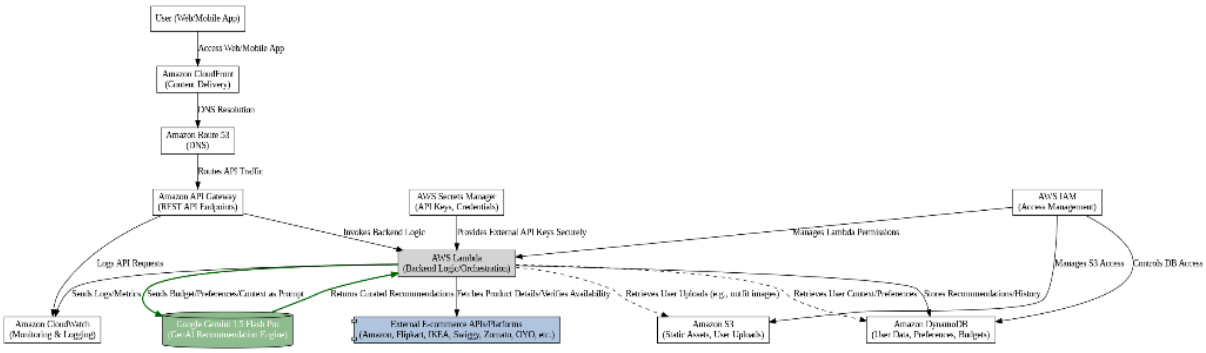
PocketSmart AI is an intelligent, modular platform designed to generate personalized, budget-aware recommendations across domains like home interiors, party planning, and jewelry selection. It integrates a lightweight Flask backend with Google's Gemini 1.5 Flash Pro, a powerful multimodal AI model that processes user inputs (text, numbers, and images) to deliver smart recommendations.

## Core Technologies

- Flask: Backend service layer for request handling, routing, and session control.
- Gemini 1.5 Flash Pro: AI model used for understanding budget context, analyzing images, generating structured suggestions, and recommending platform-specific products.
- Third-party APIs (Amazon, Flipkart, IKEA, Zomato, etc.): Product and service data sourcing.
- Frontend (HTML/CSS + JS): Interactive user interface for input collection and AI output display.

## Component-Wise Architecture

| Component | Description |
|---|---|
| Frontend UI | Web interface built with HTML/CSS allows users to input budgets, preferences, and images. Displays tailored AI-driven recommendations for each category. |
| Flask Backend | Manages user sessions, routes (/generate-home, /generate-party, /generate-jewelry), and handles communication between UI and Gemini AI. |
| Gemini AI Layer | Processes inputs using Gemini 1.5 Flash Pro to: |




## Pre-requisites

1. IBM Cloud Account Setup: IBM Cloud Console
2. Watsonx Access and Configuration: Watsonx Overview
3. Python Basics: Python.org
4. FastAPI Framework: FastAPI Docs

## Project Workflow

1. Gemini AI Setup & Initialization

- Activity 1.1: Set up Google Cloud access (or platform) with Gemini 1.5 Flash Pro API access.

- Activity 1.2: Configure API keys, usage limits, and test integration for multimodal support (text + image).

- Activity 1.3: Fine-tune prompts for budget interpretation and product recommendation generation.

2. Core Functionality Development

- Activity 2.1: Design Flask-based backend structure for routing, session control, and planner modules.

- Activity 2.2: Create endpoints for Home Planner, Party Planner, and Jewelry Planner.

- Activity 2.3: Integrate Gemini prompts and templates for each planner with support for text + optional image inputs.

3. Backend Implementation (app.py)

- Activity 3.1: Define Flask routes like /generate-home, /generate-party, /generate-jewelry.
- Activity 3.2: Organize backend using modular structure:
    - routes/: Endpoints
    - services/: Recommendation logic & AI calls
    - models/: Input/output schemas
- Activity 3.3: Setup CORS headers for frontend communication and implement basic session management.
- Activity 3.4: Include mock API calls or simulated scraping from Amazon, IKEA, Zomato, etc.

4. UI Development

- Activity 4.1: Build responsive HTML forms for:
    - Home Interior Planner (room details, quantities)
    - Party Planner (event type, budget, guests)
    - Jewelry Planner (budget, occasion, outfit image upload)
- Activity 4.2: Add dynamic result display and formatting for recommendations using JavaScript.

5. Testing & Optimization

- Activity 5.1: Perform testing with real-world use cases across all three planners.
- Activity 5.2: Evaluate Gemini response quality, budget adherence, and platform accuracy.

- Activity 5.3: Optimize prompts and add validations to prevent input edge-case failures.

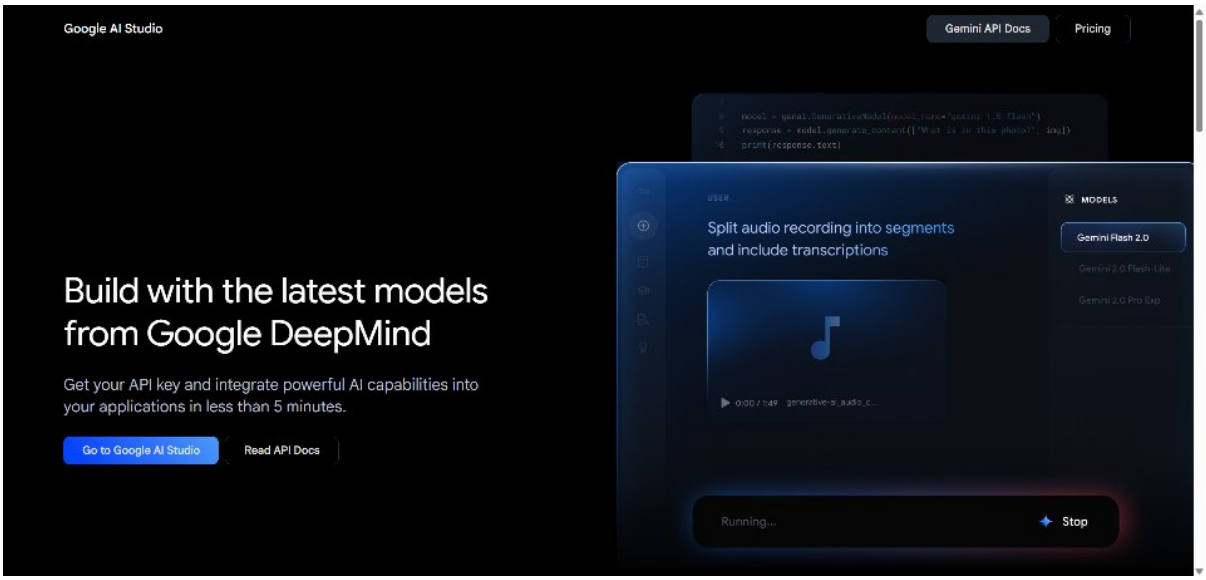
- Activity 5.4: Add fallback/default recommendations when AI returns insufficient results.

## MILESTONE 1: Gemini AI Initialization

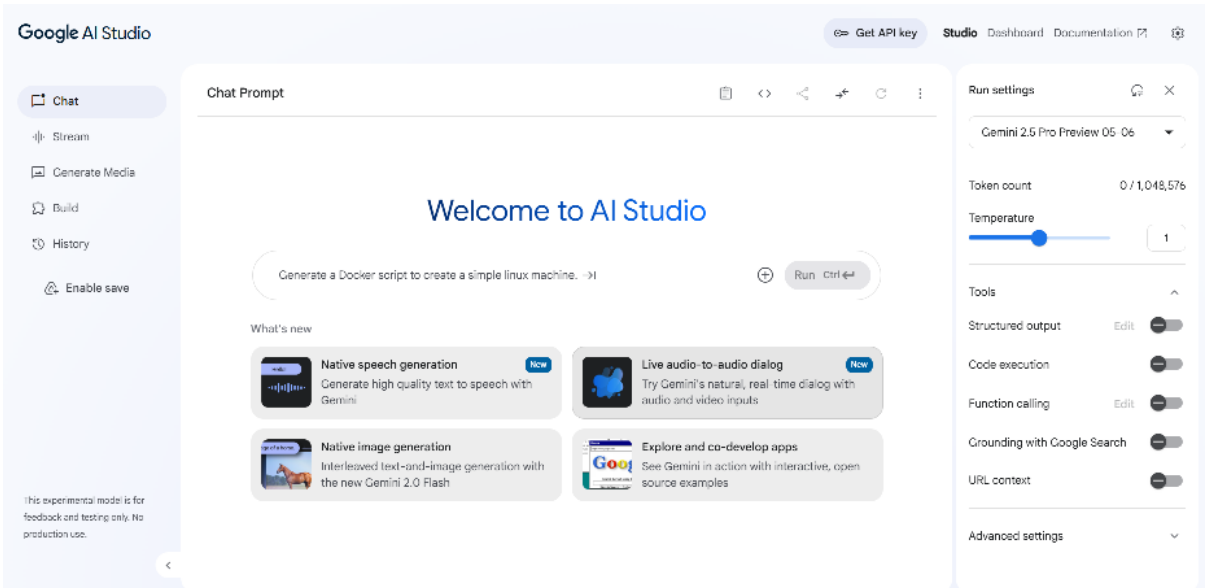
In this foundational stage, the objective is to establish and configure the core AI infrastructure required to power PocketSmart AI. The platform relies on Gemini 1.5 Flash Pro—Google's multimodal foundation model—to interpret user inputs, process budgets, and generate personalized recommendations. This milestone ensures that the development environment is equipped with authenticated access to Gemini's APIs and is capable of performing both text-based reasoning and image analysis (specifically for the Jewelry Planner).Proper completion of this setup is essential, as it enables all AI-driven functionalities like budget understanding, category-wise recommendations, and multimodal analysis to operate seamlessly.

### Activity 1.1: Set up Google Cloud account and enable Gemini API

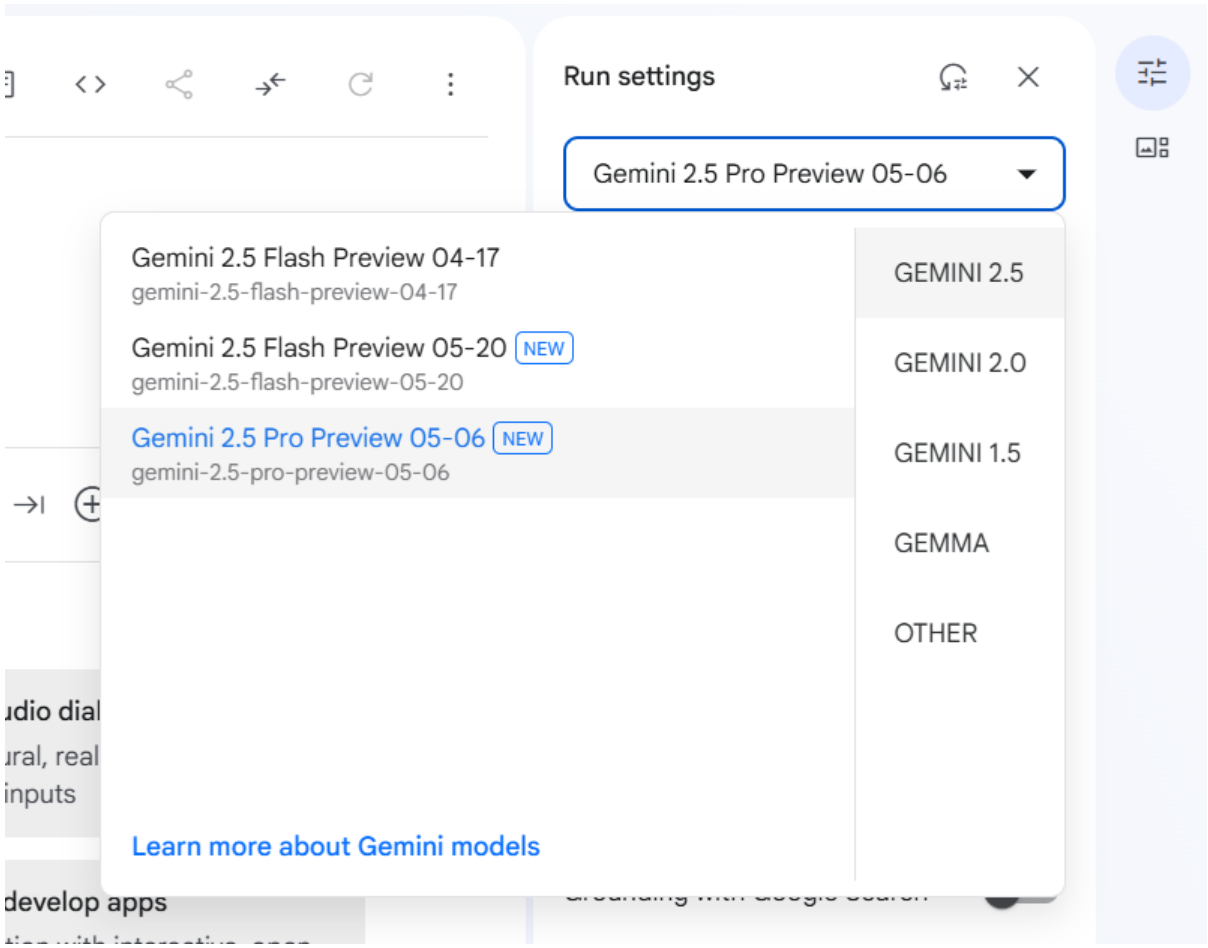
- Sign in or create a Google Cloud account via console.cloud.google.com.




- Accept all terms and conditions and click on Get API key




- You will get trending models here you can select the model here or you can set the model by go to the API Documentation.

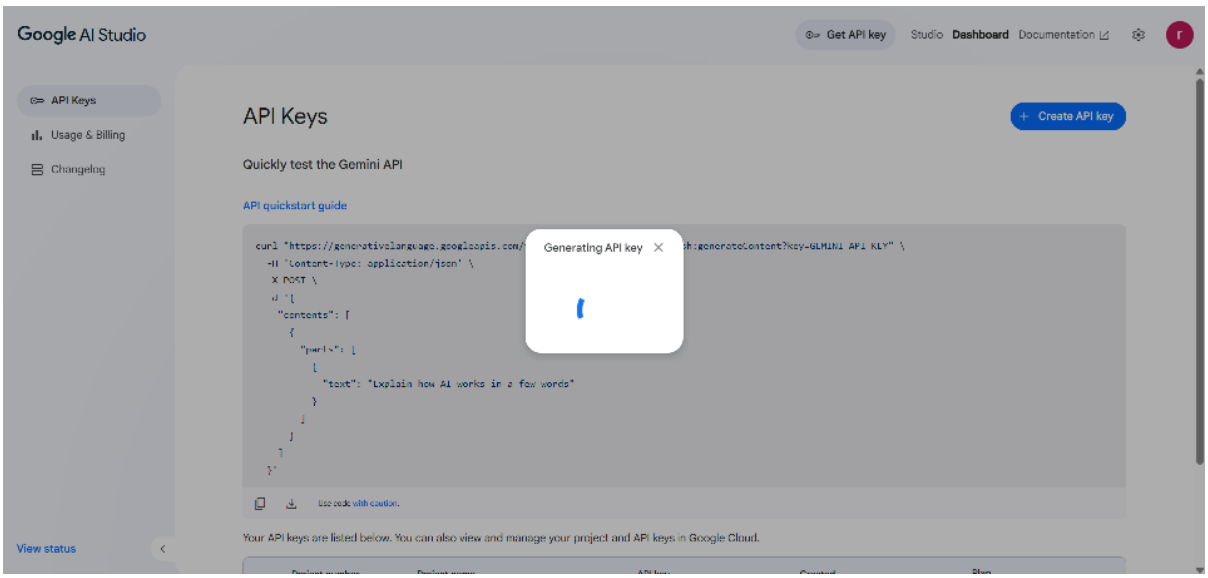

Run settings
Gemini 2.5 Pro Preview 05-06
Gemini 2.5 Flash Preview 04-17
gemini-2.5-flash-preview-04-17
Gemini 2.5 Flash Preview 05-20 NEW
gemini-2.5-flash-preview-05-20
Gemini 2.5 Pro Preview 05-06 NEW
gemini-2.5-pro-preview-05-06
GEMINI 2.5
GEMINI 2.0
GEMINI 1.5
GEMMA
OTHER
Learn more about Gemini models
develop apps

Activity 1.2: Configure access and permissions

- You will get interface like this and now click on Create API key




- Once click on API key you will the api key and make sure to copy the API key



- Activity 1.3: Validate Gemini API connectivity

  - Make a sample prompt call using Python to confirm integration.

```python
from google import genai
from google.genai import types
from PIL import Image
from io import BytesIO
import base64

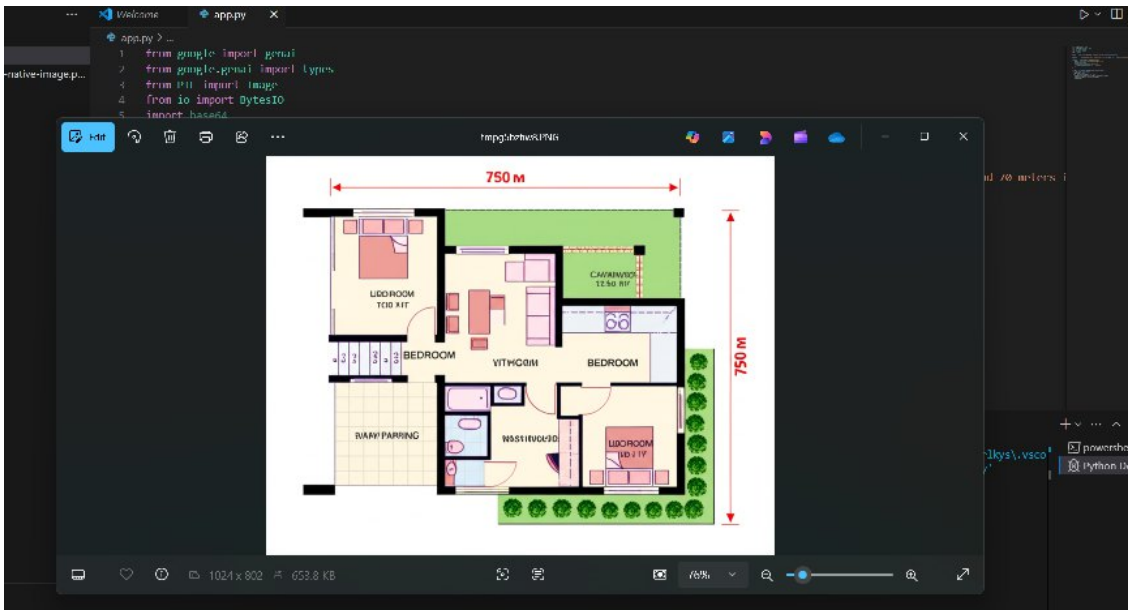
client = genai.Client(api_key="AIzaSyDdzTxAIisLXlQ")

contents = ('Generate a detailed 2D architectural floor plan for a modern two story house with a total area of 150 meters in length and 70 meters

response = client.models.generate_content(
    model="gemini-2.0-flash-exp-image-generation",
    contents=contents,
    config=types.GenerateContentConfig(
      response_modalities=['Text', 'Image']
    )
)

for part in response.candidates[0].content.parts:
  if part.text is not None:
    print(part.text)
  elif part.inline_data is not None:
    image = Image.open(BytesIO((part.inline_data.data)))
    image.save('gemini native image.png')
    image.show()
```

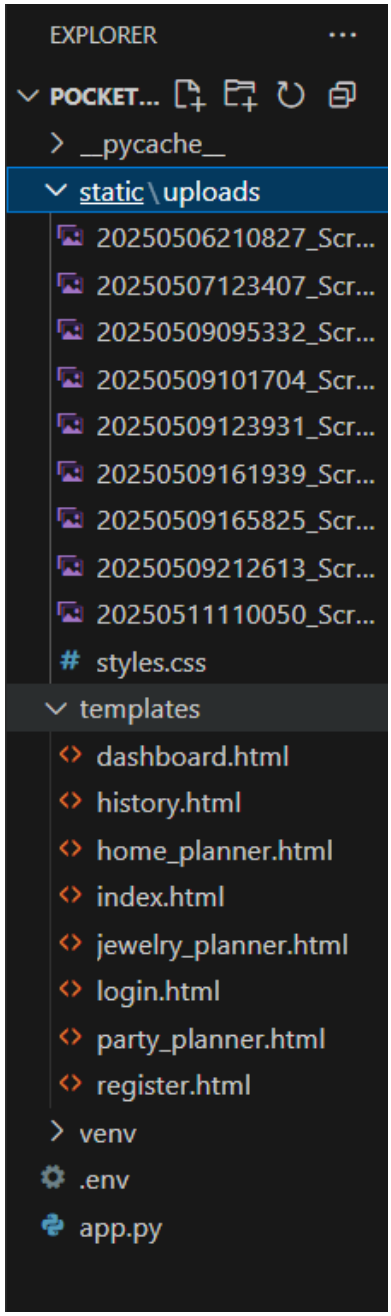
  - Test both text-only and image + text prompts to ensure multimodal compatibility.



MILESTONE 2: Core Functionalities Development

This milestone focuses on building the core backend intelligence of PocketSmart AI using the Flask framework and Google's Gemini 1.5 Flash Pro foundation model. The primary goal is to develop a robust and modular system capable of accepting user budgets (and optional image inputs), intelligently interpreting the context, and returning domain-specific, platform-aware recommendations.The backend handles prompt generation, API communication with Gemini, modular planning logic (for Home, Party, and Jewelry), and optional fallback strategies when product matches are limited. All core logic is abstracted into a utility service file (gemini_utils.py) that handles prompt orchestration, budget formatting, domain segmentation, and image analysis for multimodal tasks.

## File Explorer Structure




Activity 2.1: Fast API Instilization and import libraries

- Import all required libraries for the project

```python
import os
import json
import re
import base64
import shutil
import urllib.parse
from fastapi import FastAPI, HTTPException, Depends, File, UploadFile, Form, Request, status, Cookie
from fastapi.responses import JSONResponse, RedirectResponse
from fastapi.middleware.cors import CORSMiddleware
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from fastapi.responses import HTMLResponse
from typing import List, Optional, Dict, Any, Union
from pydantic import BaseModel, EmailStr
from passlib.context import CryptContext
from jose import JWTError, jwt
from datetime import datetime, timedelta
from PIL import Image
from io import BytesIO
import google.generativeai as genai
from dotenv import load_dotenv
import asyncio
from typing import Dict, Any, List, Optional
import uuid
```

- Fast API  Initialization

```python
# FastAPI app initialization
app = FastAPI(title="PocketSmart: AI Budget Planner")

SECRET_KEY = os.getenv("SECRET_KEY", "your_secret_key")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30
```

- Load .env environment

```python
# Load environment variables
load_dotenv()
# Configure Google Gemini AI - Do this once, with consistent environment variable name
API_KEY = os.getenv("GOOGLE_API_KEY")
if not API_KEY:
    # Try alternative environment variable name
    API_KEY = os.getenv("GEMINI_API_KEY")

if not API_KEY:
    raise ValueError("No Google API key found in environment variables. Please set GOOGLE_API_KEY in your .env file.")

genai.configure(api_key=API_KEY)

# Password hashing context
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token", auto_error=False)  # Allow optional token

class RegisterUser(BaseModel):
    username: str
    email: str
    full_name: Optional[str] = None
    password: str

# Configure CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],  # Adjust in production
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

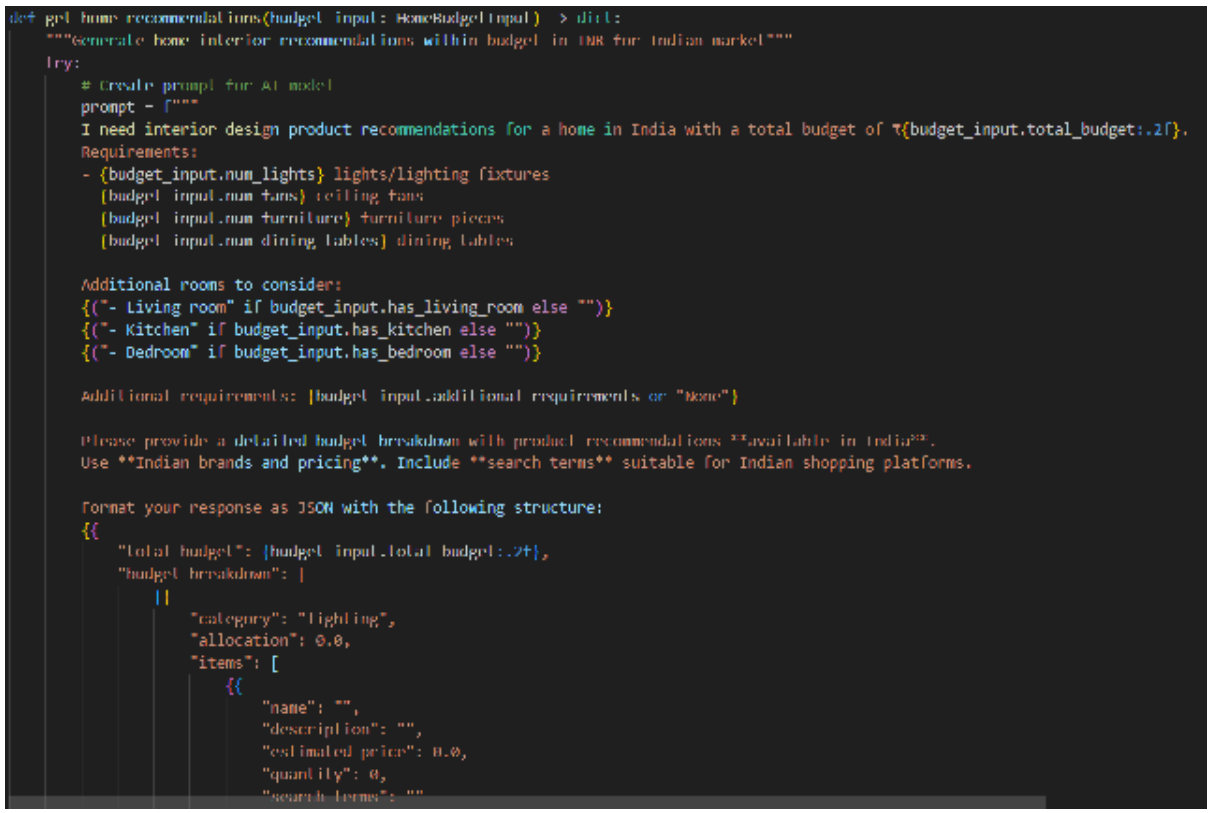
# Static files and templates
templates = Jinja2Templates(directory="templates")
os.makedirs("static/uploads", exist_ok=True)
app.mount("/static", StaticFiles(directory="static"), name="static")
```

Activity 2.2: Main Functionality code routings

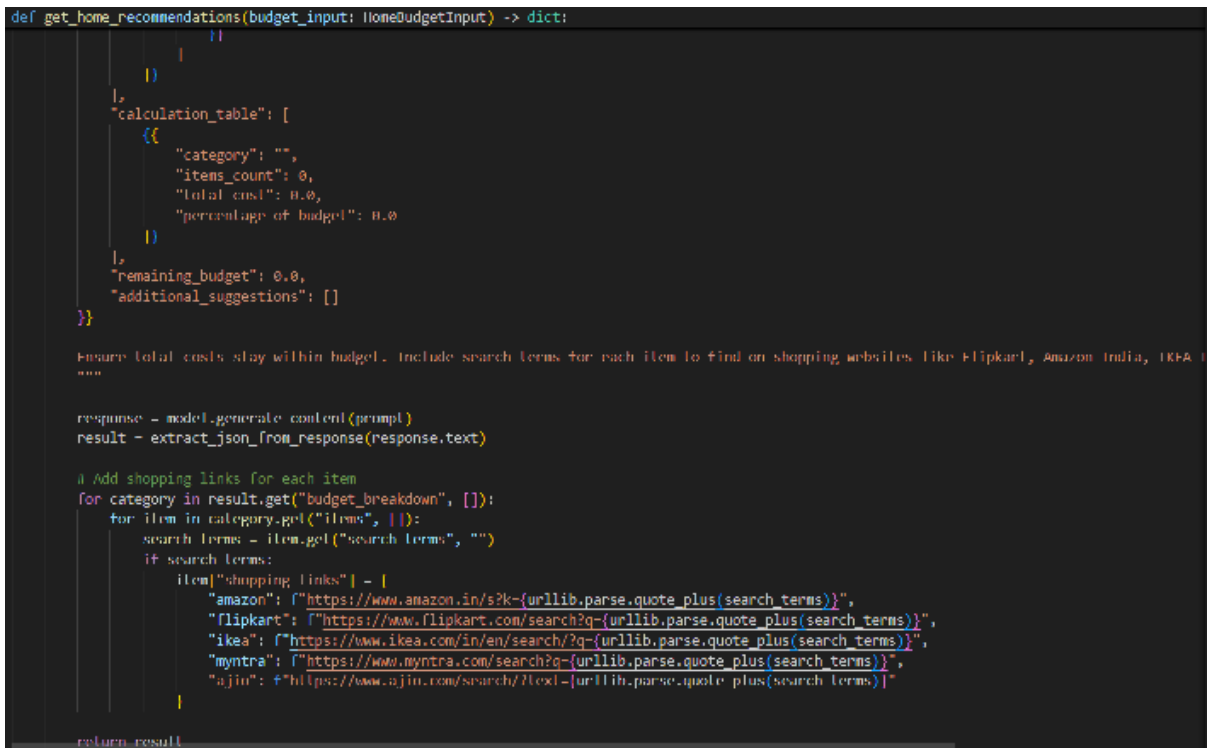
- /generate-home: Generates home interior recommendations (furniture, decor, lighting)

```python
@app.get("/home-planner", response_class=HTMLResponse)
async def home_planner(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
    """Home budget planner page"""
    return templates.TemplateResponse("home_planner.html", {"request": request, "user": current_user})
```

  - get home_recommendations by using the function



- linking the websites like amazon,flipkar,ikea ect…. to get the products



```
    except Exception as e:
        raise HTTPException(500, f"Error generating recommendations: {str(e)}")


def usd_to_inr(amount_usd: float, exchange_rate: float = 83.0) -> float:
    """Convert USD amount to INR using the specified exchange rate"""
    return amount_usd * exchange_rate
```

- /generate-party: Suggests catering, venue, and decoration options for events

```
@app.get("/party-planner", response_class=HTMLResponse)
async def party_planner(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
    """Party budget planner page"""
    return templates.TemplateResponse("party_planner.html", {"request": request, "user": current_user})
```

  - get home_recommendations by using the function

```
def get_party_recommendations(budget_input: PartyBudgetInput) -> dict:
    """Generate party planning recommendations within budget in INR for Indian market"""
    try:
        # Create prompt for AI model
        prompt = f"""
        I need party planning recommendations for India with a total budget of ₹{budget_input.total_budget:.2f}.

        Party details:
        - Type: {budget_input.party_type}
        - Number of guests: {budget_input.num_guests}
        - Venue type: {budget_input.venue_type or "Not specified"}
        - Catering needed: {"Yes" if budget_input.needs_catering else "No"}
        - Decoration needed: {"Yes" if budget_input.needs_decoration else "No"}
        - Entertainment needed: {"Yes" if budget_input.needs_entertainment else "No"}

        Additional requirements: {budget_input.additional_requirements or "None"}

        Please provide a detailed budget breakdown with specific recommendations available in India using INR prices.
        Use Indian brands, services, and typical cost expectations.

        Format your response as JSON with the following structure:
        {{
            "total_budget": {budget_input.total_budget:.2f},
            "budget_breakdown": [
                {{
                    "category": "venue",
                    "allocation": 0.0,
                    "items": [
                        {{
                            "name": "",
                            "description": "",
                            "estimated_price": 0.0,
                            "quantity": 0,
                            "search_terms": ""
                        }}
                    ]
                }}
```

```
def get_party_recommendations(budget_input: PartyBudgetInput) -> dict:
            ],
            "venue_suggestions": [
                {{
                    "name": "",
                    "type": "",
                    "capacity": 0,
                    "estimated_cost": 0.0,
                    "search_terms": ""
                }}
            ],
            "remaining_budget": 0.0,
            "additional_suggestions": []
        }}

        Ensure all costs are in INR and total does not exceed the given budget.
        Provide search terms suitable for Indian websites such as BookMyShow, Swiggy, Flipkart, etc.
        """

        response = model.generate_content(prompt)
        result = extract_json_from_response(response.text)

        # Create INR calculation table
        result["calculation_table_inr"] = []
        categories = {}

        for category in result.get("budget_breakdown", []):
            cat_name = category.get("category", "Misc")
            cat_allocation = category.get("allocation", 0)

            if cat_name not in categories:
                categories[cat_name] = {
                    "category": cat_name,
                    "items_count": 0,
                    "total_cost": 0,
                    "percentage_of_budget": 0
                }
```

- linking the websites like amazon,flipkar,ikea ect…. to get the products

```
def get_party_recommendations(budget_input: PartyBudgetInput) -> dict:
            for item in category.get("items", []):
                categories[cat_name]["items_count"] += 1
                categories[cat_name]["total_cost"] += item.get("estimated_price", 0)

            if result["total_budget"] > 0:
                categories[cat_name]["percentage_of_budget"] = (categories[cat_name]["total_cost"] / result["total_budget"]) * 100

        for cat_data in categories.values():
            result["calculation_table_inr"].append(cat_data)

        # Define relevant shopping platforms for each category
        category_platforms = {
            "venue": ["google", "booking", "makemytrip", "oyorooms", "nobroker"],
            "catering": ["swiggy", "zomato"],
            "food": ["swiggy", "zomato", "bigbasket", "amazon", "flipkart"],
            "drinks": ["swiggy", "zomato", "bigbasket", "amazon", "flipkart"],
            "decoration": ["amazon", "flipkart", "meesho", "myntra"],
            "entertainment": ["bookmyshow", "amazon", "flipkart"],
            "gifts": ["amazon", "flipkart", "myntra", "meesho"],
            "photography": ["google", "amazon", "flipkart"],
            "music": ["amazon", "flipkart", "bookmyshow"],
            "games": ["amazon", "flipkart"],
            "accessories": ["amazon", "flipkart", "myntra", "meesho"],
            "transportation": ["makemytrip", "google"],
            "return gifts": ["amazon", "flipkart", "myntra", "meesho"]
        }

        # Default platforms if category is not in our predefined list
        default_platforms = ["amazon", "flipkart", "google"]

        # Add shopping links for each item
        for category in result.get("budget_breakdown", []):
            cat_name = category.get("category", "").lower()
            relevant_platforms = category_platforms.get(cat_name, default_platforms)

            for item in category.get("items", []):
```

- using the if condition to categorize the situation based products

```
def get_party_recommendations(budget_input: PartyBudgetInput) -> dict:
        for category in result.get("budget_breakdown", []):
            cat_name = category.get("category", "").lower()
            relevant_platforms = category_platforms.get(cat_name, default_platforms)

            for item in category.get("items", []):
                search_terms = item.get("search_terms", "")
                if search_terms:
                    item["shopping_links"] = {}

                    # Only add relevant platforms for this category
                    if "amazon" in relevant_platforms:
                        item["shopping_links"]["amazon"] = f"https://www.amazon.in/s?k={urllib.parse.quote_plus(search_terms)}"
                    if "flipkart" in relevant_platforms:
                        item["shopping_links"]["flipkart"] = f"https://www.flipkart.com/search?q={urllib.parse.quote_plus(search_terms)}"
                    if "bigbasket" in relevant_platforms:
                        item["shopping_links"]["bigbasket"] = f"https://www.bigbasket.com/ps/?q={urllib.parse.quote_plus(search_terms)}"
                    if "swiggy" in relevant_platforms:
                        item["shopping_links"]["swiggy"] = f"https://www.swiggy.com/search?query={urllib.parse.quote_plus(search_terms)}"
                    if "zomato" in relevant_platforms:
                        item["shopping_links"]["zomato"] = f"https://www.zomato.com/search?q={urllib.parse.quote_plus(search_terms)}"
                    if "bookmyshow" in relevant_platforms:
                        item["shopping_links"]["bookmyshow"] = f"https://in.bookmyshow.com/search?q={urllib.parse.quote_plus(search_terms)}"
                    if "myntra" in relevant_platforms:
                        item["shopping_links"]["myntra"] = f"https://www.myntra.com/search?q={urllib.parse.quote_plus(search_terms)}"
                    if "meesho" in relevant_platforms:
                        item["shopping_links"]["meesho"] = f"https://www.meesho.com/search?q={urllib.parse.quote_plus(search_terms)}"
                    if "google" in relevant_platforms:
                        item["shopping_links"]["google"] = f"https://www.google.com/search?q={urllib.parse.quote_plus(search_terms)}"
                    if "booking" in relevant_platforms:
                        item["shopping_links"]["booking"] = f"https://www.booking.com/search.html?ss={urllib.parse.quote_plus(search_terms)}"
                    if "makemytrip" in relevant_platforms:
                        item["shopping_links"]["makemytrip"] = f"https://www.makemytrip.com/hotels/hotel-listing/?searchText={urllib.parse.quote
                    if "oyorooms" in relevant_platforms:
                        item["shopping_links"]["oyorooms"] = f"https://www.oyorooms.com/search/?location={urllib.parse.quote_plus(search_terms)}"
                    if "nobroker" in relevant_platforms:
                        item["shopping_links"]["nobroker"] = f"https://www.nobroker.in/property/search?searchTerm={urllib.parse.quote_plus(search
```



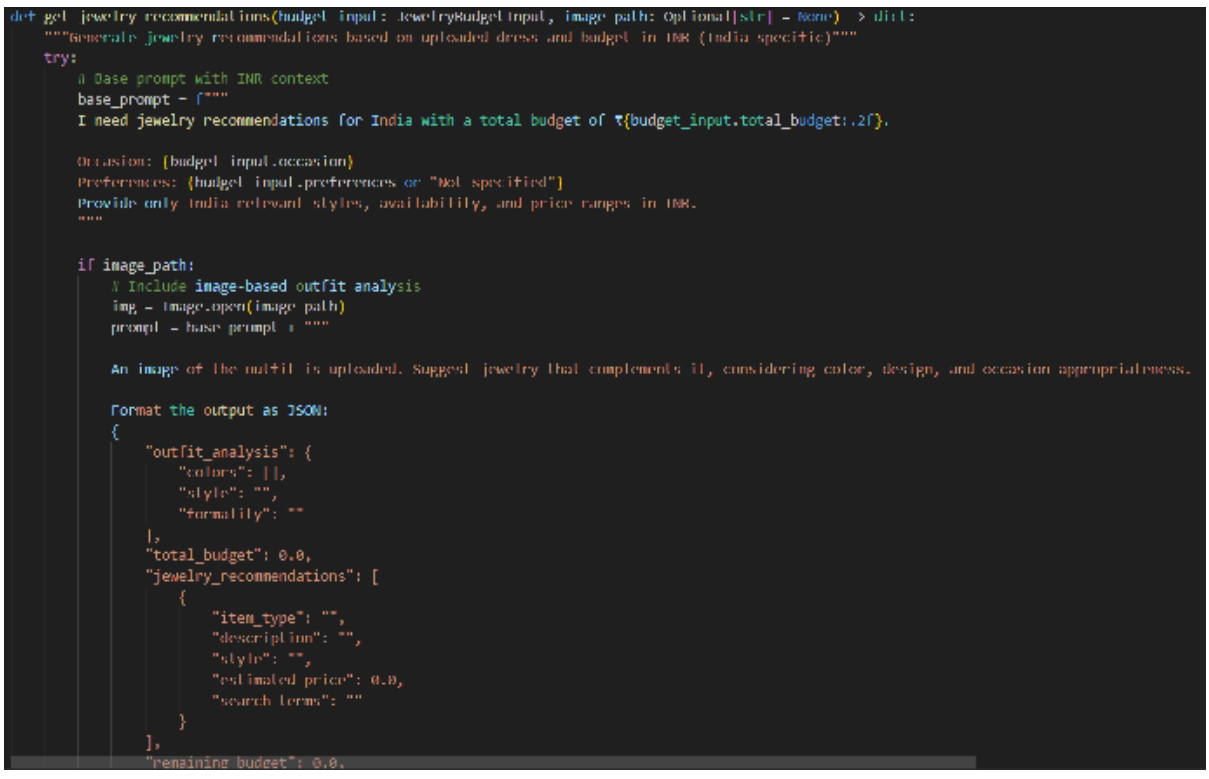
```
    # Add search links for venue suggestions
    venue_platforms = ["google", "booking", "makemytrip", "oyorooms", "nobroker"]
    for venue in result.get("venue_suggestions", []):
        search_terms = venue.get("search_terms", "")
        if search_terms:
            venue["search_links"] = {}
            if "google" in venue_platforms:
                venue["search_links"]["google"] = f"https://www.google.com/search?q={urllib.parse.quote_plus(search_terms)}"
            if "booking" in venue_platforms:
                venue["search_links"]["booking"] = f"https://www.booking.com/search.html?ss={urllib.parse.quote_plus(search_terms)}"
            if "makemytrip" in venue_platforms:
                venue["search_links"]["makemytrip"] = f"https://www.makemytrip.com/hotels/hotel-listing/?searchText={urllib.parse.quote_plus(s
            if "oyorooms" in venue_platforms:
                venue["search_links"]["oyorooms"] = f"https://www.oyorooms.com/search/?location={urllib.parse.quote_plus(search_terms)}"
            if "nobroker" in venue_platforms:
                venue["search_links"]["nobroker"] = f"https://www.nobroker.in/property/search?searchTerm={urllib.parse.quote_plus(search_terms

    return result

except Exception as e:
    raise HTTPException(500, f"Error generating recommendations: {str(e)}")
```

- /generate-jewelry: Accepts text + image inputs to return style-matched jewelry options.
- async def jewelry_planner(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
- """Jewelry budget planner page"""
- return templates.TemplateResponse("jewelry_planner.html", {"request": request, "user": current_user})

- get home_recommendations by using the function



```
def get_jewelry_recommendations(budget_input: JewelryBudgetInput, image_path: Optional[str] = None) -> dict:
    """Generate jewelry recommendations based on uploaded dress and budget in INR (India specific)"""
    try:
        # Base prompt with INR context
        base_prompt = f"""
        I need jewelry recommendations for India with a total budget of ₹{budget_input.total_budget:.2f}.

        Occasion: {budget_input.occasion}
        Preferences: {budget_input.preferences or "Not specified"}
        Provide only India-relevant styles, availability, and price ranges in INR.
        """

        if image_path:
            # Include image-based outfit analysis
            img = Image.open(image_path)
            prompt = base_prompt + """

            An image of the outfit is uploaded. Suggest jewelry that complements it, considering color, design, and occasion appropriateness.

            Format the output as JSON:
            {
                "outfit_analysis": {
                    "colors": [],
                    "style": "",
                    "formality": ""
                },
                "total_budget": 0.0,
                "jewelry_recommendations": [
                    {
                        "item_type": "",
                        "description": "",
                        "style": "",
                        "estimated_price": 0.0,
                        "search_terms": ""
                    }
                ],
                "remaining_budget": 0.0,
```

```
def get_jewelry_recommendations(budget_input: JewelryBudgetInput, image_path: Optional[str] = None) -> dict:
            "styling_tips": []
        }

        Make sure prices are in INR and stay within budget.
        Include Indian-friendly search terms for shopping.
        """
        response = model.generate_content([prompt, img])
    else:
        # Only text input
        prompt = base_prompt + """

        Format the output as JSON:
        {
            "total_budget": 0.0,
            "jewelry_recommendations": [
                {
                    "item_type": "",
                    "description": "",
                    "style": "",
                    "estimated_price": 0.0,
                    "search_terms": ""
                }
            ],
            "remaining_budget": 0.0,
            "styling_tips": []
        }

        Keep prices in INR and relevant to Indian brands.
        """
        response = model.generate_content(prompt)

    result = extract_json_from_response(response.text)

    # Add shopping links for each item (India-specific)
    for item in result.get("jewelry_recommendations", []):
        search_terms = item.get("search_terms", "")
```

- linking the websites like amazon,flipkar,ikea ect…. to get the products

```
def get_jewelry_recommendations(budget_input: JewelryBudgetInput, image_path: Optional[str] = None) -> dict:
    # Add shopping links for each item (India-specific)
    for item in result.get("jewelry_recommendations", []):
        search_terms = item.get("search_terms", "")
        if search_terms:
            item["shopping_links"] = {
                "amazon": f"https://www.amazon.in/s?k={urllib.parse.quote_plus(search_terms)}",
                "flipkart": f"https://www.flipkart.com/search?q={urllib.parse.quote_plus(search_terms)}",
                "bluestone": f"https://www.bluestone.com/search.html?query={urllib.parse.quote_plus(search_terms)}",
                "tanishq": f"https://www.tanishq.co.in/search?q={urllib.parse.quote_plus(search_terms)}",
                "caratlane": f"https://www.caratlane.com/search?q={urllib.parse.quote_plus(search_terms)}",
                "melorra": f"https://www.melorra.com/search?q={urllib.parse.quote_plus(search_terms)}",
                "meesho": f"https://www.meesho.com/search?q={urllib.parse.quote_plus(search_terms)}"
            }

    return result

except Exception as e:
    raise HTTPException(500, f"Error generating recommendations: {str(e)}")
```

Activity 2.3: Login and Register page routings

- Login page

  /login: Authenticates user credentials and initiates a session upon successful login.

```python
@app.get("/login", response_class=HTMLResponse)
async def login_page(request: Request):
    """Serve the login page"""
    # Check if user is already logged in
    try:
        token = await get_token(request)
        if token:
            user = await get_current_user(request, token)
            if user:
                return RedirectResponse(url="/dashboard", status_code=status.HTTP_302_FOUND)
    except Exception:
        pass

    return templates.TemplateResponse("login.html", {"request": request})
```

- Register Page

  /register: Handles new user registration by accepting and securely storing user credentials.

```python
@app.get("/register", response_class=HTMLResponse)
async def register_page(request: Request):
    """Serve the registration page"""
    # Check if user is already logged in
    try:
        token = await get_token(request)
        if token:
            user = await get_current_user(request, token)
            if user:
                return RedirectResponse(url="/dashboard", status_code=status.HTTP_302_FOUND)
    except Exception:
        pass

    return templates.TemplateResponse("register.html", {"request": request})
```

- Logout Page

/logout: Terminates the current user session and redirects to the login page.

```python
@app.post("/logout")
async def logout(request: Request):
    """Logout user by blacklisting their token and clearing session"""
    token = await get_token(request)

    if token:
        # Add token to blacklist
        blacklisted_tokens.add(token)

        try:
            # Extract username from token to remove from active sessions
            payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
            username = payload.get("sub")
            if username and username in active_sessions:
                del active_sessions[username]
        except JWTError:
            pass

    # Use RedirectResponse instead of JSONResponse for proper redirection
    response = RedirectResponse(url="/login", status_code=status.HTTP_302_FOUND)
    response.delete_cookie(key="access_token")
```

Activity 2.4: Ensure modular code structure

- /token: Issues a secure JWT token after successful authentication, enabling authorized access to protected API routes.

```python
@app.post("/token", response_model=Token)
async def login_for_access_token(form_data: OAuth2PasswordRequestForm = Depends()):
    """Login endpoint to get access token"""
    user = authenticate_user(users_db, form_data.username, form_data.password)
    if not user:
        raise HTTPException(
            status_code=401,
            detail="Incorrect username or password",
            headers={"WWW-Authenticate": "Bearer"},
        )

    # Create access token
    access_token_expires = timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    access_token = create_access_token(
        data={"sub": user.username}, expires_delta=access_token_expires
    )

    # Create or update session for the user
    existing_user_data = {}
    if user.username in active_sessions:
        existing_user_data = active_sessions[user.username].user_data
        # If there's an old token, blacklist it
        old_token = active_sessions[user.username].token
        blacklisted_tokens.add(old_token)

    active_sessions[user.username] = UserSession(
        username=user.username,
        login_time=datetime.utcnow(),
        last_activity=datetime.utcnow(),
        token=access_token,
        user_data=existing_user_data
    )

    # Return response with cookie
    response = JSONResponse(content={"access_token": access_token, "token_type": "bearer"})

    response.set_cookie(
        key="access_token",
        value=access_token,  # Store token directly without Bearer prefix
        httponly=True,
        max_age=ACCESS_TOKEN_EXPIRE_MINUTES * 60,
        samesite="lax"
    )

    return response
```

- /session-info: Retrieves metadata about the current user session, such as user ID and login status.
- /session-data: Returns detailed session-specific data used for personalization and recommendation tracking.

```python
@app.get("/session-info")
async def get_session_info(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
    """Get current user's session information"""
    if current_user.username in active_sessions:
        session = active_sessions[current_user.username]
        return {
            "username": session.username,
            "login_time": session.login_time,
            "last_activity": session.last_activity,
            "session_duration": (datetime.utcnow() - session.login_time).total_seconds() // 60,  # in minutes
            "user_data": session.user_data
        }
    else:
        raise HTTPException(status_code=404, detail="No active session found")

@app.post("/session-data")
async def update_session_data(
    data: Dict[str, Any],
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Update user's session data"""
    if current_user.username in active_sessions:
        active_sessions[current_user.username].user_data.update(data)
        active_sessions[current_user.username].last_activity = datetime.utcnow()
        return {"message": "Session data updated", "data": active_sessions[current_user.username].user_data}
    else:
        raise HTTPException(status_code=404, detail="No active session found")
# Fix for the /home-budget endpoint to properly handle the form submission
```

MILESTONE 3: Backend – FastAPI Integration (main.py)

This milestone focuses on implementing the FastAPI-powered backend for PocketSmart AI. It acts as the bridge between the frontend interface and the Gemini-based AI recommendation

engine defined in gemini_utils.py. The backend is designed with modular architecture, providing clean and maintainable API endpoints for planners across Home, Party, and Jewelry domains.It also includes features like CORS setup for cross-origin requests, session handling for personalized user experience, and structured input/output models to ensure smooth AI interaction and reliable frontend rendering.

- Activity 3.1: Define FastAPI Routes

  - Home Planner (/generate-home): Accepts home-related preferences and budget to return tailored product suggestions.



```python
@app.post("/home-budget")
async def plan_home_budget(
    budget_input: HomeBudgetInput,
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Generate home budget recommendations"""
    # Store last budget planning in session data
    if current_user.username in active_sessions:
        active_sessions[current_user.username].user_data["last_home_budget"] = {
            "timestamp": datetime.utcnow().isoformat(),
            "budget": budget_input.total_budget,
            "requirements": {
                "lights": budget_input.num_lights,
                "fans": budget_input.num_fans,
                "furniture": budget_input.num_furniture,
                "dining_tables": budget_input.num_dining_tables
            }
        }

    # Get recommendations
    result = get_home_recommendations(budget_input)

    # Save to history
    save_to_history(
        username=current_user.username,
        recommendation_type="home",
        input_data=budget_input.dict(),
        result=result
    )

    return result
```

  - Party Planner (/generate-party): Processes party details and guest count to recommend venue, food, and decoration items.

```python
# Update the party-budget endpoint
@app.post("/party-budget")
async def plan_party_budget(
    budget_input: PartyBudgetInput,
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Generate party budget recommendations"""
    # Store last budget planning in session data
    if current_user.username in active_sessions:
        active_sessions[current_user.username].user_data["last_party_budget"] = {
            "timestamp": datetime.utcnow().isoformat(),
            "budget": budget_input.total_budget,
            "party_type": budget_input.party_type,
            "guests": budget_input.num_guests
        }

    # Get recommendations
    result = get_party_recommendations(budget_input)

    # Save to history
    save_to_history(
        username=current_user.username,
        recommendation_type="party",
        input_data=budget_input.dict(),
        result=result
    )

    return result
```

- Jewelry Planner (/generate-jewelry): Uses text and optional image inputs to suggest jewelry based on outfit and occasion.

```python
# Update the jewelry-budget endpoint
@app.post("/jewelry-budget")
async def plan_jewelry_budget(
    total_budget: float = Form(...),
    occasion: str = Form(...),
    preferences: Optional[str] = Form(None),
    image: Optional[UploadFile] = File(None),
    request: Request = None,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Generate jewelry budget recommendations with optional outfit image"""
    # Create the budget input model
    budget_input = JewelryBudgetInput(
        total_budget=total_budget,
        occasion=occasion,
        preferences=preferences
    )

    image_path = None
    if image:
        image_path = save_upload_file(image)

    # Store last budget planning in session data
    if current_user.username in active_sessions:
        active_sessions[current_user.username].user_data["last_jewelry_budget"] = {
            "timestamp": datetime.utcnow().isoformat(),
            "budget": budget_input.total_budget,
            "occasion": budget_input.occasion,
            "has_image": image is not None
        }

    # Get recommendations
    result = get_jewelry_recommendations(budget_input, image_path)

    # Save to history with image info
    input_data = budget_input.dict()
    if image:
        input_data["image"] = image.filename

    save_to_history(
        username=current_user.username,
        recommendation_type="jewelry",
        input_data=input_data,
        result=result
    )

    return result
@app.get("/recommendation-history")
async def get_recommendation_history(
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Get the user's recommendation history"""
    if current_user.username not in user_recommendations:
```

```
    return {"history": []}

# Sort history by timestamp (newest first)
history = sorted(
    user_recommendations[current_user.username],
    key=lambda x: x.timestamp,
    reverse=True
)

# Convert to dict for JSON response
history_data = []
for item in history:
    history_data.append({
        "id": item.id,
        "timestamp": item.timestamp,
        "type": item.recommendation_type,
        "input": item.input_summary,
        "summary": item.result_summary
    })

return {"history": history_data}
```

- User Auth (/register, /login, /logout, /token): Manages user registration, authentication, and session control.

- Session Info (/session-info, /session-data): Retrieves user-specific session data for personalized AI interaction.

- Activity 3.2: Modular Architecture Setup

/recommendations-details: Returns detailed AI-generated product recommendations based on user budget, preferences, and selected category (Home, Party, or Jewelry).

```python
# Step 6: Add an endpoint to get details for a specific recommendation
@app.get("/recommendation-details/{recommendation_id}")
async def get_recommendation_details(
    recommendation_id: str,
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Get the full details of a specific recommendation"""
    if current_user.username not in user_recommendations:
        raise HTTPException(status_code=404, detail="No recommendations found")

    # Find the recommendation with the given ID
    for item in user_recommendations[current_user.username]:
        if item.id == recommendation_id:
            return {
                "id": item.id,
                "timestamp": item.timestamp,
                "type": item.recommendation_type,
                "input": item.input_summary,
                "full_result": item.full_result
            }

    raise HTTPException(status_code=404, detail="Recommendation not found")
```

- Activity 3.3: CORS & Static Routing

/history: Retrieves the user's past recommendation queries and results for review or re-use.

```python
# Step 7: Add a new endpoint to serve the history page
@app.get("/history", response_class=HTMLResponse)
async def history_page(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
    """History page to view past recommendations"""
    return templates.TemplateResponse("history.html", {"request": request, "user": current_user})
```

- Activity 3.4: Startup and main function

/startup: Initializes essential application services and loads configuration settings when the FastAPI server starts.

```python
@app.on_event("startup")
async def setup_session_cleanup():
    """Background task to clean up expired sessions"""
    async def cleanup_expired_sessions():
        while True:
            current_time = datetime.utcnow()
            # Check for sessions that have been inactive for more than 30 minutes
            expired_sessions = [
                username for username, session in active_sessions.items()
                if (current_time - session.last_activity).total_seconds() > 1800  # 30 minutes
            ]

            # Remove expired sessions
            for username in expired_sessions:
                if username in active_sessions:
                    print(f"Removing expired session for {username}")
                    del active_sessions[username]

            # Wait for 5 minutes before checking again
            await asyncio.sleep(300)

    # Start the background task
    asyncio.create_task(cleanup_expired_sessions())
```

__main__: Entry point of the FastAPI application that starts the server using uvicorn when the script is run directly.

```python
# Main entry point
if __name__ == "__main__":
    import uvicorn
    print("Starting PocketSmart: AI Budget Planner...")
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

## MILESTONE 4: UI Development

This milestone focuses on developing a lightweight, responsive, and intuitive frontend for PocketSmart AI using HTML, CSS, and Jinja2 templates. The interface enables users to interact seamlessly with the system's budget-based recommendation capabilities across Home Interior, Party Planning, and Jewelry categories. It is tightly integrated with FastAPI backend endpoints to facilitate real-time input and AI-powered output rendering.The goal is to maintain a clean, user-friendly design that supports multi-category inputs, displays structured AI recommendations, and includes authentication and session-based personalization.

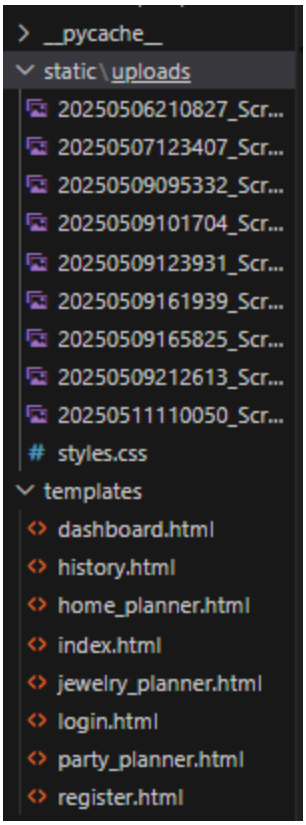
### Activity 4.1: Develop HTML Frontend Interface

- Design a home page with budget input fields and category selectors (Home, Party, Jewelry)

- Implement individual forms for each planner module connected to FastAPI POST routes

- Display AI responses in well-structured, card-like layouts (e.g., product suggestions, item details, prices)

Templates Directory Structure:




MILESTONE 5: Testing & Optimization

This milestone focuses on validating the reliability, accuracy, and contextual awareness of PocketSmart AI's recommendation engine. The system is tested using real-world user scenarios across different planning categories—Home Interiors, Party Planning, and Jewelry Suggestions—to ensure consistent performance. Optimization efforts include refining AI prompt structures, improving session handling, and ensuring a smooth UI/UX across devices.

Activity 5.1: Test with Real-World Budget Inputs Across Domains

Conduct end-to-end testing using varied budgets and user preferences for:

- Home decor setups from platforms like IKEA, Amazon

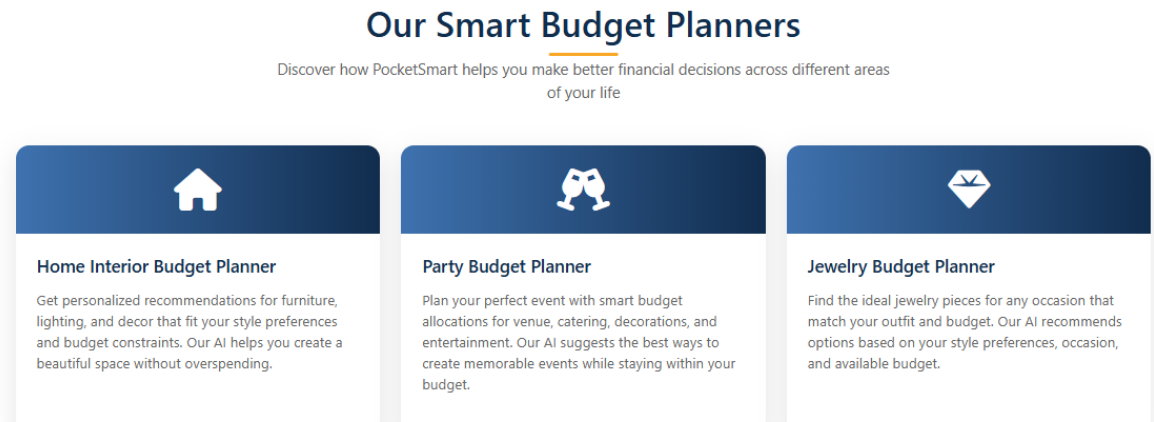
- Party packages including food (Swiggy/Zomato), venues (OYO), and decor

- Jewelry suggestions matched with outfit styles, occasions, and image inputs

- Home Page

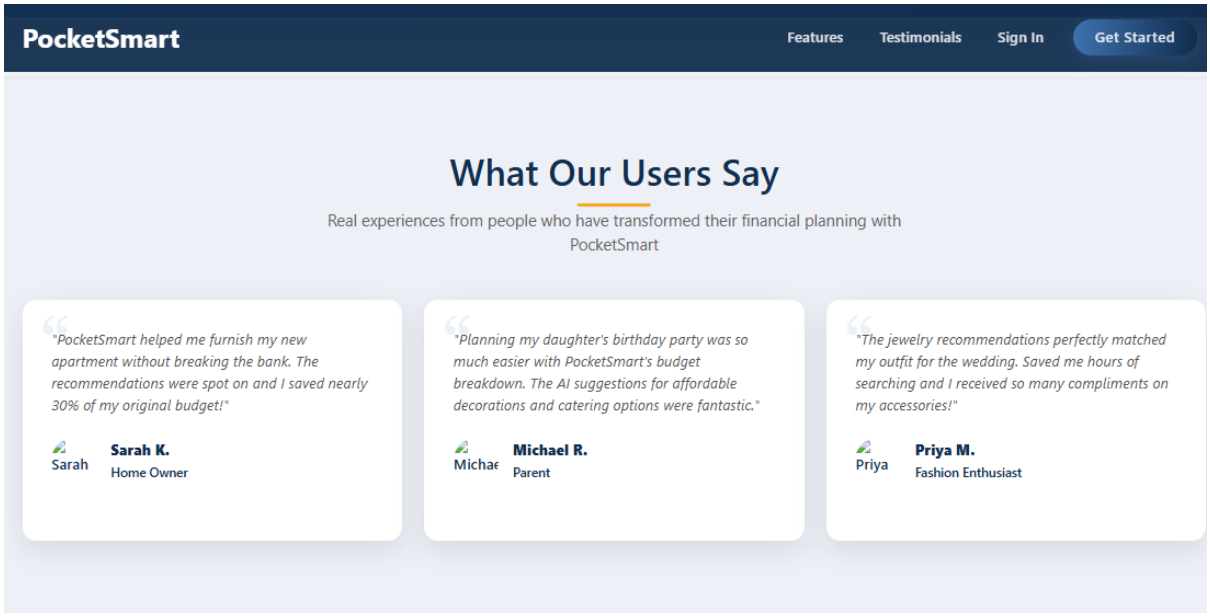
Home Page: The main landing page introducing PocketSmart AI's features with links to planner modules and user actions.

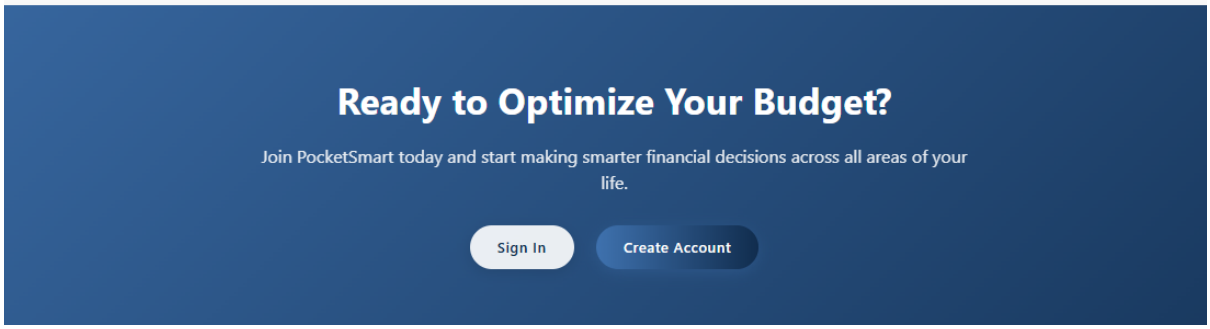


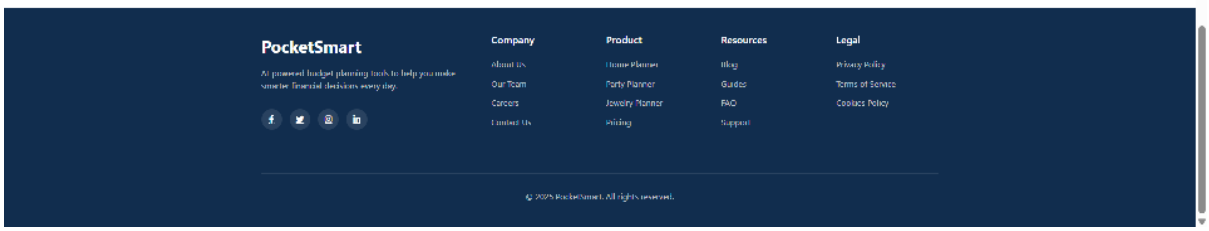

- Testimonials Page

Testimonials Page: Showcases real user reviews and success stories to demonstrate the platform's effectiveness.



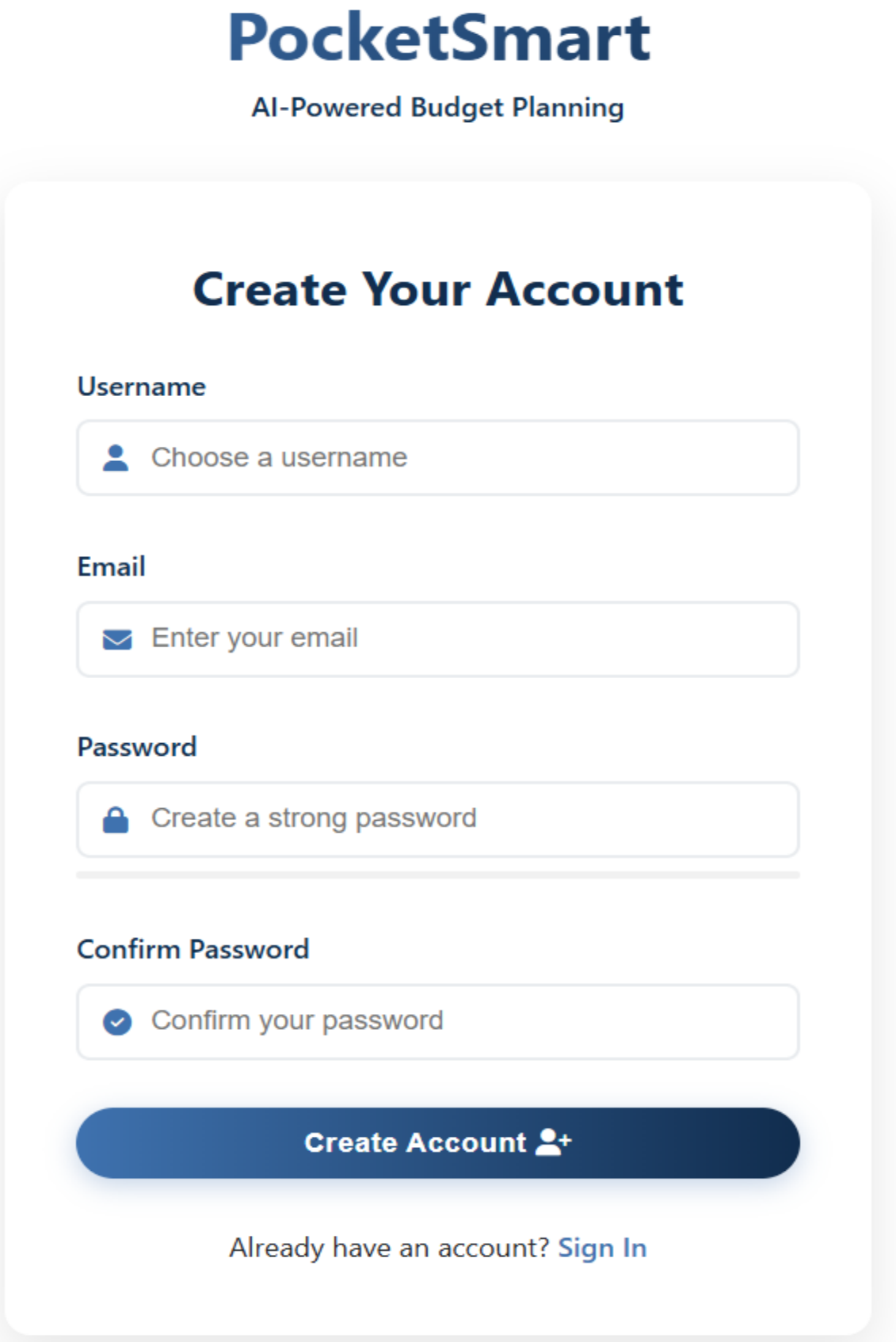




- Footer Part



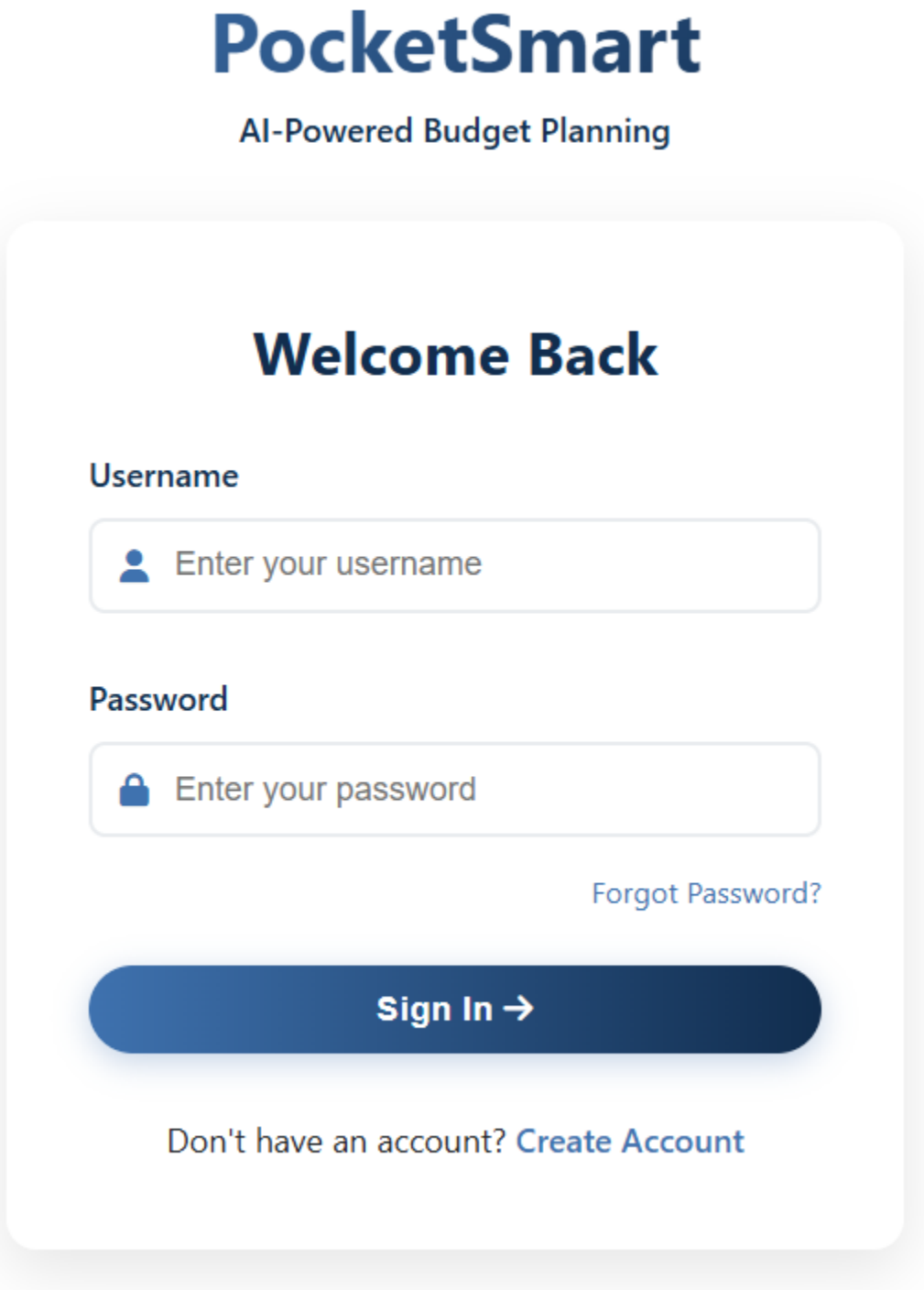

- Register Page

Register Page: Allows new users to create an account to access personalized budget recommendations.




- Login Page

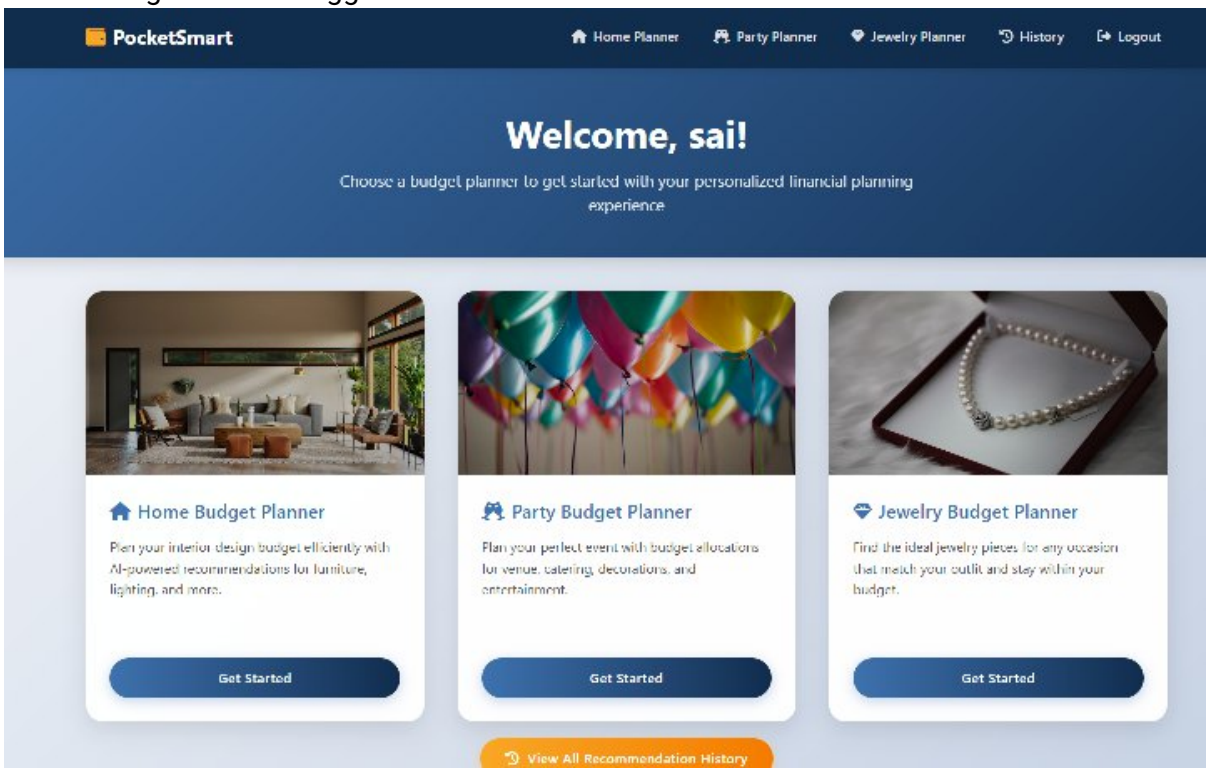
Login Page: Authenticates existing users to access their dashboard and saved data.

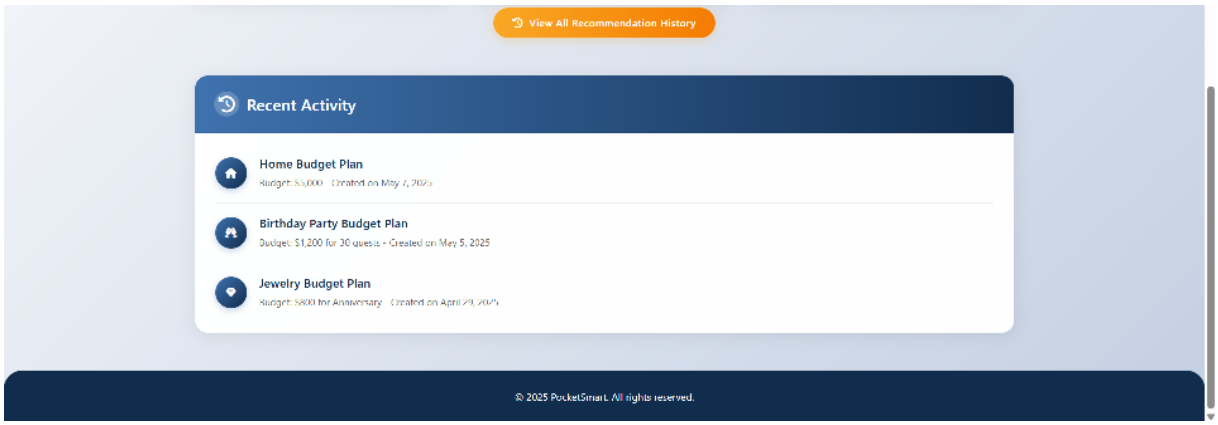



- User Dashboard

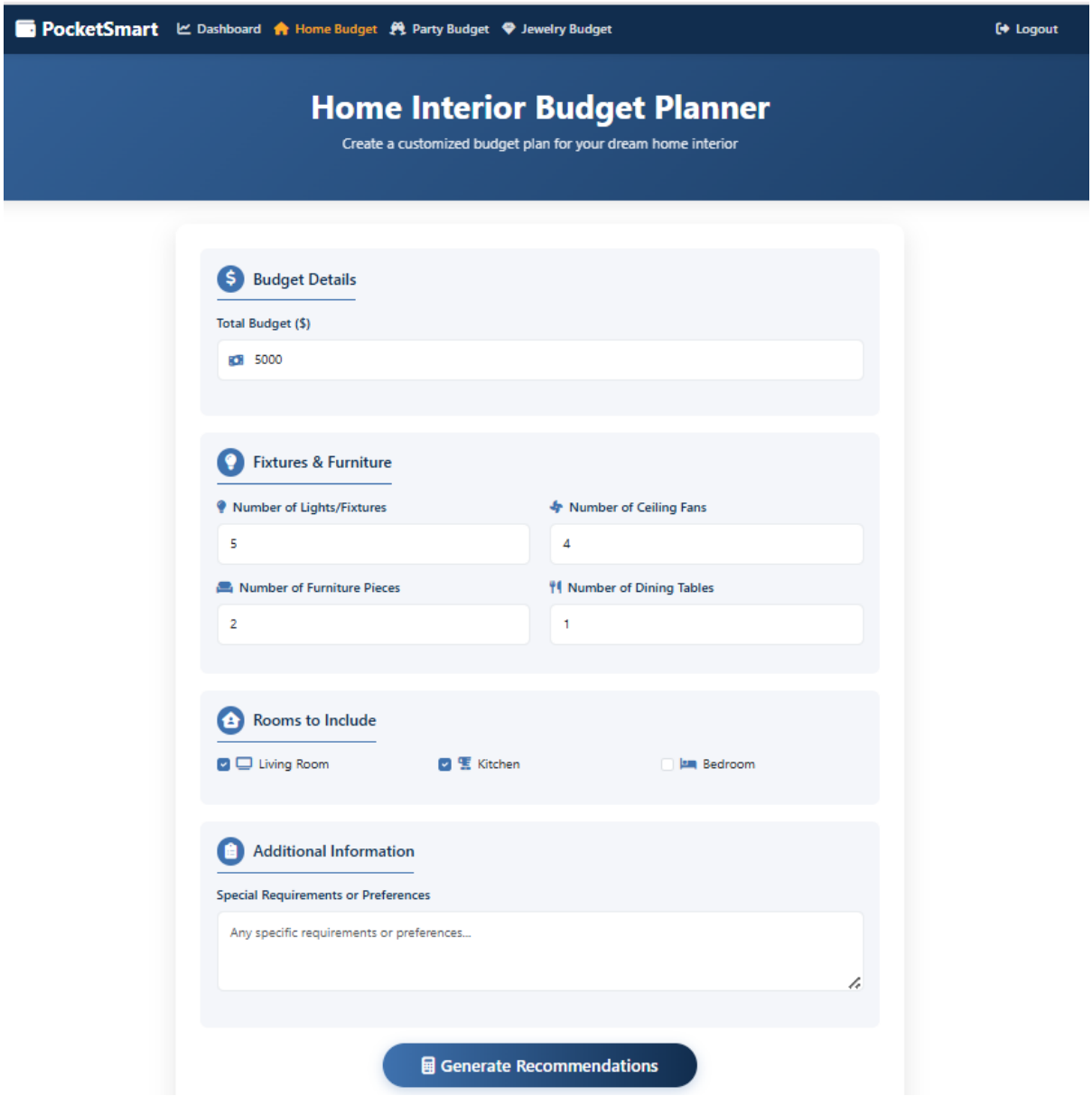
User Dashboard: Displays recent recommendations, saved queries, and personalized

insights for the logged-in user.

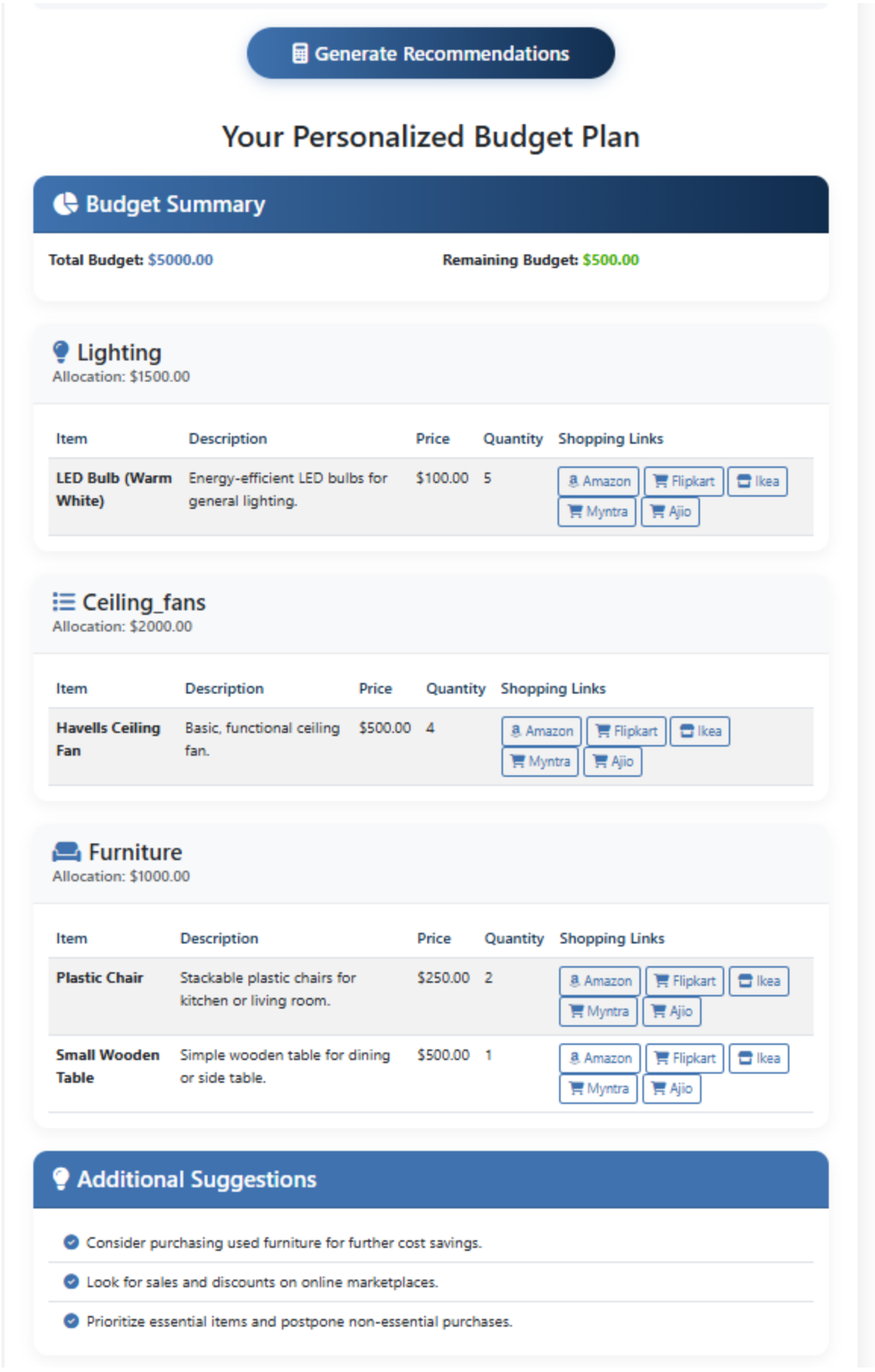



Home Interior Budget Planner Page: Lets users input home decor preferences and budget to get tailored suggestions from multiple platforms.

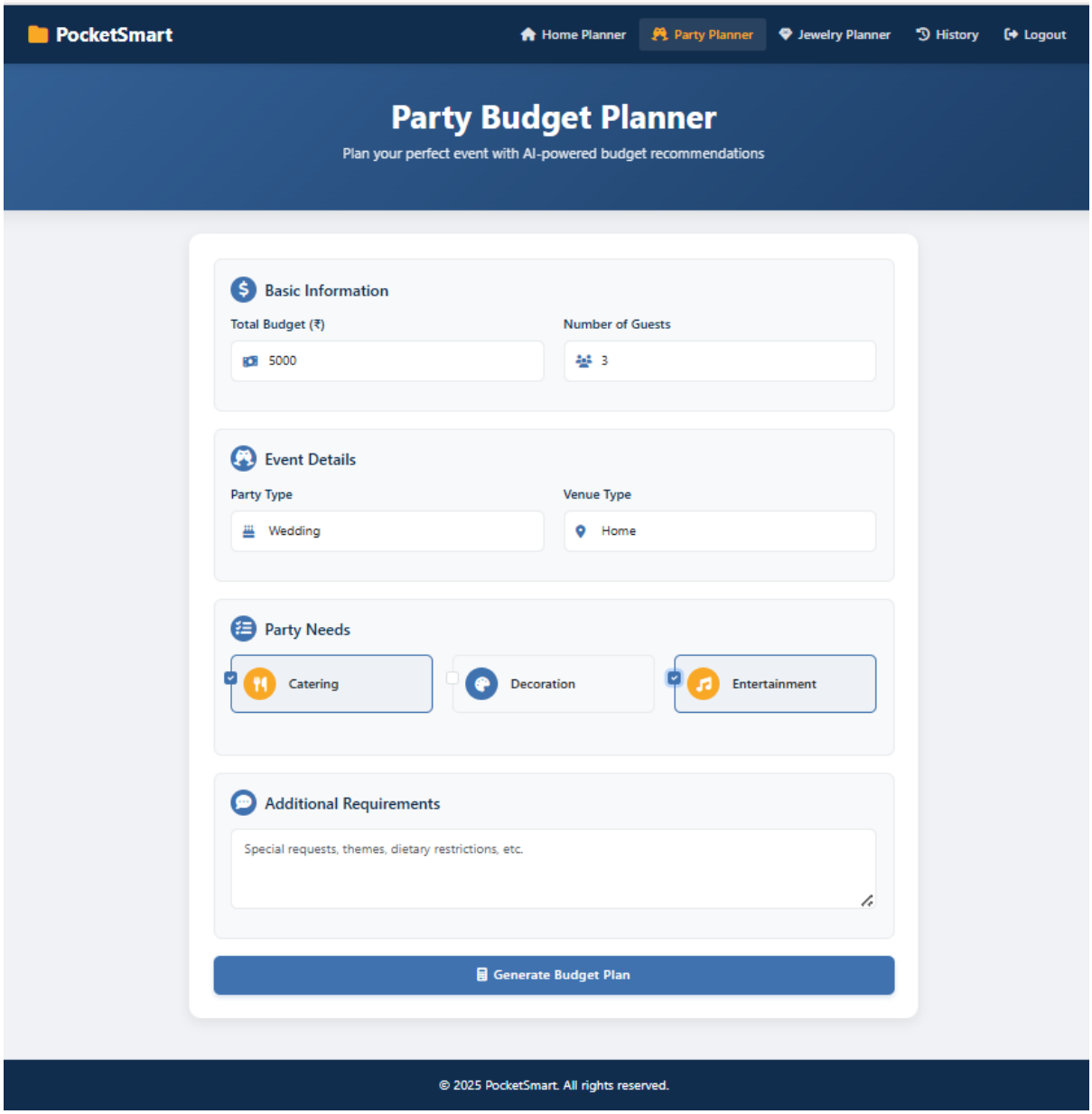



Home Interior Recommendations Page: Displays AI-generated product suggestions for home decor based on the user's budget and style preferences.




Party Budget Planner Page: Helps users plan events by suggesting venues, food, and

decorations based on budget and guest count




Party Budget Recommendations Page: Shows curated party planning options, including food, venues, and decorations, matched to the provided budget.

# Your Party Budget Plan

Print/Save

## Budget: ₹5000.00

### Venue — ₹0.00

**Home** — ₹0.00

Utilizing the home as the venue.

Shop on: Google, Booking, Makemytrip, Oyorooms, Nobroker

### Catering — ₹2000.00

**Home-cooked meal** — ₹2000.00

Simple, home-cooked meal for 3 people.

Shop on: Swiggy, Zomato

### Entertainment — ₹2000.00

**Streaming service subscription** — ₹500.00

One-time purchase of a movie or renting a few movies on a platform like Netflix, Amazon Prime Video, or Hotstar.

Shop on: Amazon, Flipkart, Bookmyshow

**Board games/cards** — ₹500.00

Purchase of a new board game or card game.

Shop on: Amazon, Flipkart, Bookmyshow

**Music playlist** — ₹0.00

Creating a celebratory playlist on Spotify or YouTube Music.

Shop on: Amazon, Flipkart, Bookmyshow

**Small gift for couple** — ₹1000.00

A small token of appreciation for the couple.

Shop on: Amazon, Flipkart, Bookmyshow

### Contingency — ₹1000.00

**Unexpected expenses** — ₹1000.00

Buffer for unforeseen costs.

Shop on: Amazon, Flipkart, Google

| | |
|---|---|
| **Total Budget** | **₹5000.00** |
| Allocated | ₹5000.00 |
| Remaining | ₹0.00 |

## Venue Suggestions

**Home** — ₹0.00

Type: Residential

Location: undefined

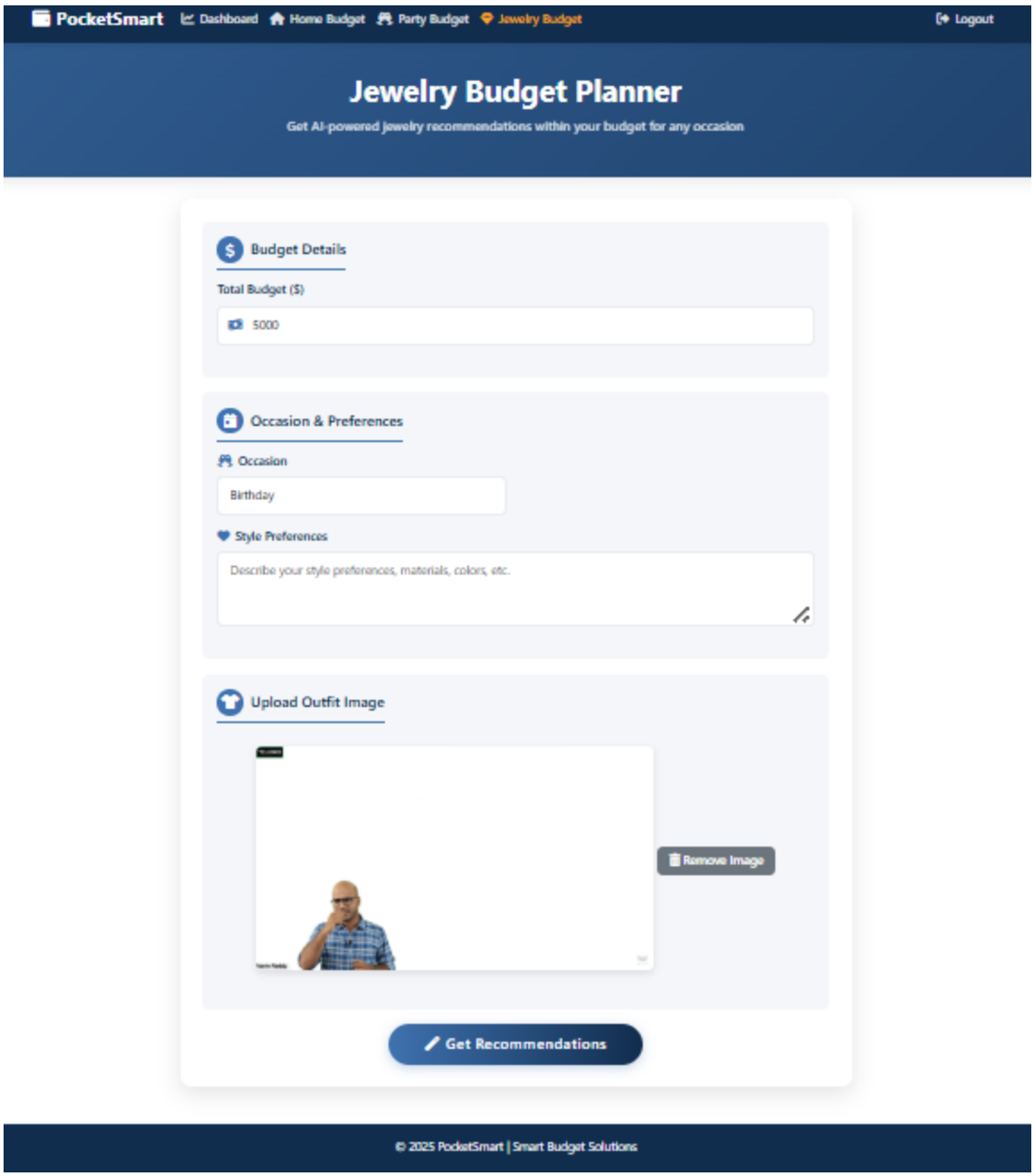
undefined

Website, View on Map

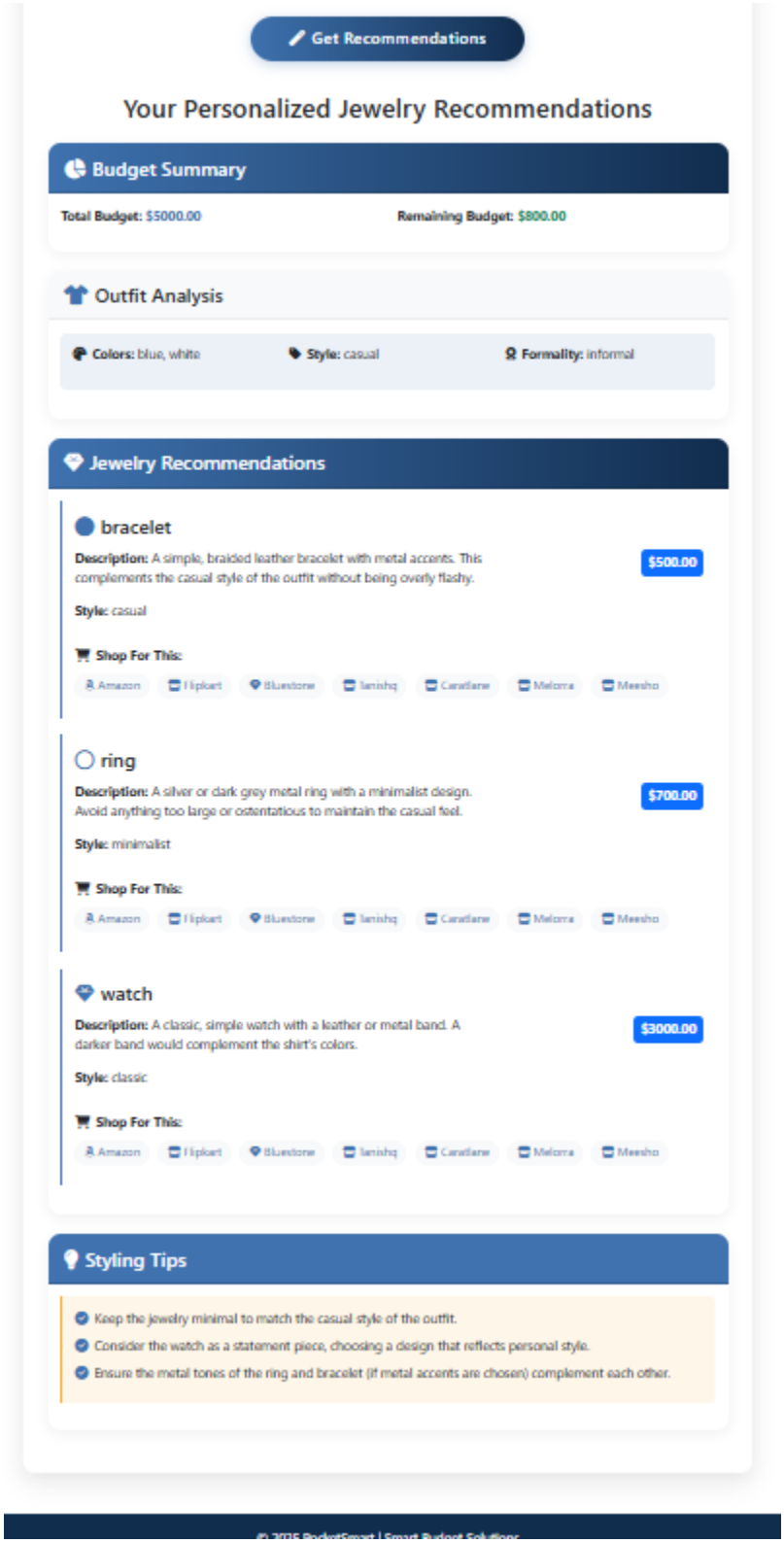
## Additional Suggestions

- Consider making the meal a potluck style if comfortable with guests to reduce catering costs.
- Look for discounts or offers on streaming services or board games.
- Homemade decorations can be a cost-effective alternative if you decide to add some.

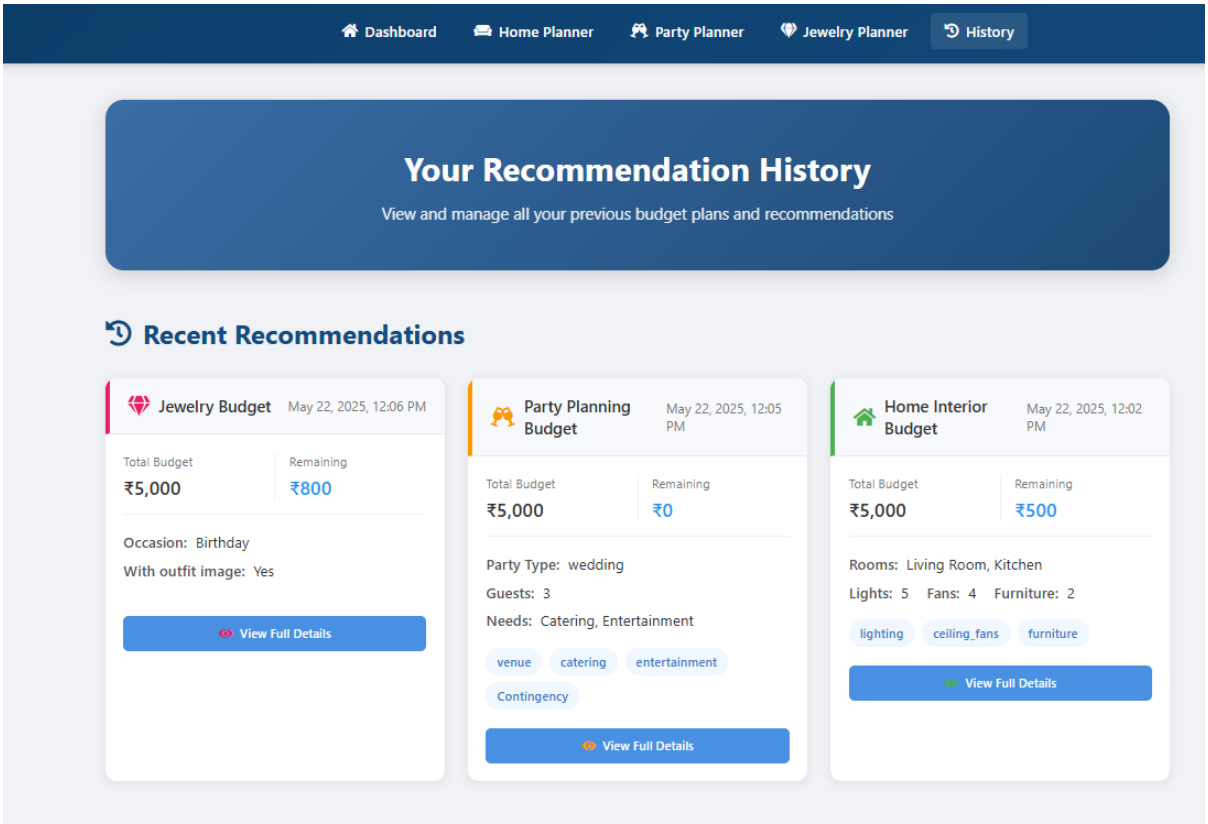
Jewelry Budget Planner Page: Provides jewelry recommendations aligned with occasion and outfit style, using text and optional image inputs.




Jewelry Budget Recommendations Page: Presents personalized jewelry options tailored to the occasion, outfit, and budget, using Gemini-generated insights.




Recommendation History Page: Displays a log of the user's past recommendation queries and results, allowing easy review and reuse of previous plans.




aq

## Conclusion

PocketSmart AI redefines the way individuals plan budgets for everyday lifestyle needs by

delivering smart, AI-driven recommendations across home interiors, party planning, and jewelry selection. By combining the power of FastAPI with Gemini 1.5 Flash Pro, the system intelligently processes user inputs—budgets, preferences, and even images—to generate accurate and context-aware suggestions from multiple trusted platforms like Amazon, Flipkart, IKEA, Swiggy, Zomato,  OYO and more.From backend architecture to AI integration and a responsive frontend, PocketSmart AI has been built with a focus on efficiency, scalability, and user-centric design. The FastAPI backend ensures modular, secure routing and robust session handling, while the Jinja2-powered frontend offers an intuitive and clean interface for users to interact with the system in real time.

Each module—from registration to recommendations—is designed to simplify decision-making and enhance the shopping or planning experience through automation and personalization. Whether it’s finding the right wall art within budget or organizing a last-minute party, PocketSmart AI acts as a smart digital assistant, saving time and effort while optimizing outcomes.With its seamless fusion of generative AI and web technologies, PocketSmart AI stands as a practical example of how AI can enhance everyday decision-making, bringing intelligent recommendations to users' fingertips—budget-friendly, fast, and personalized.